

Polimorfismo y principios básicos de diseño

M4: Fundamento de programación en Java

|AE6: Utilizar principios básicos de diseño orientado a objetos para la implementación de una pieza de software acorde al lenguaje Java para resolver un problema de baja complejidad

Introducción

En esta lección conoceremos cómo implementar el polimorfismo tanto a través de la herencia de clases como mediante interfaces.

El **polimorfismo** es la capacidad de un objeto de comportarse de diferentes maneras según el contexto en el que se utilice. A través de la herencia de clases, podemos crear una jerarquía de clases donde una clase hereda atributos y comportamientos de otra. Esto nos permite crear instancias de una clase padre y utilizarlas de manera polimórfica, es decir, invocar métodos específicos de las clases hijas.

Además de la **herencia**, también podemos implementar el polimorfismo mediante interfaces. Una **interfaz** define un conjunto de métodos que una clase puede implementar, lo que nos permite establecer un contrato y garantizar un comportamiento consistente en diferentes clases que implementan esa interfaz. Esto nos brinda una mayor flexibilidad y nos permite programar contra interfaces en lugar de clases concretas.

También exploraremos los principios básicos de **diseño orientado a objetos**. Estos principios son pautas generales que nos ayudan a escribir un código más limpio, estructurado y mantenible. Aprenderemos sobre la importancia de la responsabilidad única, la cohesión y el acoplamiento, y cómo estos conceptos afectan la calidad y la reutilización del código.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Utilizar principios básicos de diseño orientado a objetos para la implementación de una pieza de software acorde al lenguaje Java para resolver un problema de baja complejidad

Herencia

La herencia es un pilar importante de la POO. Es el mecanismo mediante el cual **una clase es capaz de heredar todas las características (atributos y métodos) de otra clase.**

Las propiedades comunes se definen en la **superclase** (clase padre/madre) y las **subclases** heredan estas propiedades (Clase hija/o). En esta relación, la frase "Un objeto es un-tipo-de una superclase" debe tener sentido, por ejemplo: un perro es un tipo de animal, o también, una heladera es un tipo de electrodoméstico.

La herencia apoya el concepto de "**reutilización**", es decir, cuando deseamos crear una nueva clase y ya existe una clase que incluye parte del código que queremos, podemos reutilizar los campos y métodos de la clase existente.

La manera de usar herencia es a través de la palabra **extends**.

```
public class SubClase extends SuperClase {  
  
    // Contenido de la clase ;  
  
}
```

Atributos y Constructores

```
public class Vehiculo {  
  
    protected int id;  
    protected String tipo;  
    protected String marca;  
  
}  
  
public class Coche extends Vehiculo{  
  
    private String color;  
  
}
```

En la superclase podemos observar que los atributos están creados con el modificador de acceso **protected** y no private. Esto es porque el modificador de acceso protected permite que las subclases puedan acceder a los atributos de la superclase sin la necesidad de getters y setters.

Una de las principales diferencias entre los constructores y los métodos es que los constructores no se heredan, pero los métodos sí. Todos los constructores

definidos en una superclase pueden ser usados desde constructores de las subclases a través de la palabra reservada **super**. La palabra clave **super** es la que me permite elegir qué constructor, entre los que tiene definida la clase padre, es el que debo usar.

Si la superclase tiene definido el constructor vacío y no colocamos una llamada explícita **super**, se llamará el constructor vacío de la superclase.

```
public class Coche extends Vehiculo{  
    private String color;  
  
    public Coche(String tipo, String marca, int id, String color) {  
        super(tipo, marca, id);  
  
        this.color = color;  
    }  
}
```

La palabra clave **super** nos sirve para hacer referencia o llamar a los atributos, métodos y constructores de la superclase en las subclases.

super.atributoClasePadre;

super.metodoClasePadre;

Herencia y métodos

Los métodos heredados pueden ser redefinidos en las clases hijas. Este mecanismo se denomina **sobreescritura**. La sobreescritura permite a las clases hijas utilizar un método definido en la superclase.

Una subclase sobreescribe un método de su superclase cuando define un método con las mismas características (nombre, número y tipo de argumentos) que el método de la superclase. Las subclases emplean la sobreescritura de métodos la mayoría de las veces para agregar o modificar la funcionalidad del método heredado de la clase padre.

La sobreescritura permite que las clases hijas sumen sus métodos en torno al funcionamiento, y esto se logra escribiendo la anotación **@Override** arriba del método que queremos sobreescribir. El método debe llamarse igual en la subclase como en la superclase.

Polimorfismo

El polimorfismo hace referencia a la cualidad por la que un objeto adquiere o puede tener diferentes formas. En Java, el **polimorfismo** alude al modo en que **se pueden crear y utilizar dos o más métodos con el mismo nombre para ejecutar funciones diferentes**.

Polimorfismo mediante herencia

Imagina que tienes una mascota. Puede ser un perro, un gato o un pájaro. Cada uno de ellos tiene sus propias características y comportamientos específicos, como ladrar, maullar o volar. Ahora, piensa en cómo podrías representar todas estas mascotas en un programa de Java.

En lugar de crear una clase separada para cada tipo de mascota, podemos utilizar el polimorfismo para tratar a todas las mascotas de manera general. Esto significa que podemos tener una clase genérica llamada "Mascota" y luego crear subclases como "Perro", "Gato" y "Pájaro" que heredan de la clase "Mascota".

La magia del polimorfismo ocurre cuando creamos un arreglo o una lista de mascotas. Podemos agregar instancias de diferentes subclases en la misma lista y tratarlas como si fueran del tipo genérico "Mascota". Esto significa que podemos llamar a métodos comunes de la clase "Mascota", como "hacerSonido()", sin saber qué tipo específico de mascota estamos manipulando.

Cuando llamamos al método "hacerSonido()" en una mascota particular, Java se encarga de invocar el comportamiento correspondiente según el tipo de mascota real. Por ejemplo, si la mascota es un perro, se ladrará; si es un gato, se maullará; y así con todas las mascotas que agregues.

Esto nos brinda una gran flexibilidad y nos permite escribir código genérico que puede trabajar con diferentes tipos de mascotas sin preocuparnos por los detalles específicos de cada uno.

Clases abstractas

Una clase es **abstracta** cuando no se permiten instancias de esa clase, es decir, no se pueden crear objetos.

Usualmente las clases abstractas suelen ser las superclases, esto sucede porque creemos que la superclase o clase padre no debería poder instanciarse. Por ejemplo, si tenemos una clase Animal, el usuario no debería poder crear un Animal, sino que sólo debería poder instanciar objetos de las subclases.

```
public abstract class Animal { }
```

Cuando una clase posee al menos un método abstracto, esa clase necesariamente debe ser marcada como abstracta.

Métodos abstractos

Un método abstracto es un método declarado pero no implementado, es decir, es un método del que solo se escribe su nombre, parámetros, y tipo devuelto, pero no su código de implementación. Estos métodos se heredan y se sobrescribe por las clases hijas quienes son las responsables de implementar sus funcionalidades.

Interfaces

Una interfaz se compone de un conjunto de declaraciones de cabeceras de métodos (sin implementar, de forma similar a un método abstracto) que especifican un protocolo de comportamiento para una o varias clases. Además, una clase puede implementar una o varias interfaces: en ese caso, la clase debe proporcionar la declaración y definición de todos los métodos de cada una de las interfaces o bien declararse como clase abstracta. Por otro lado, una interfaz puede emplearse también para declarar constantes que luego puedan ser utilizadas por otras clases.

Por otro lado se ha de decir que una Interface no se pueden definir **atributos** salvo que estos sean **estáticos o constantes**; es decir, "**static**" o "**final**".

Una interfaz puede parecer similar a una clase abstracta, pero existen una serie de diferencias entre una interfaz y una clase abstracta:

- Todos los métodos de una interfaz se declaran implícitamente como abstractos y públicos.
- Una clase abstracta no puede implementar los métodos declarados como abstractos, una interfaz no puede implementar ningún método (ya que todos son abstractos).
- Una interfaz no declara variables de instancia.
- Una clase puede implementar varias interfaces, pero sólo puede tener una clase ascendiente directa.
- Una clase abstracta pertenece a una jerarquía de clases mientras que una interfaz no pertenece a una jerarquía de clases.

```
public interface IdentificadorInterfaz {
    // Cuerpo de la interfaz ...
}
```

Para declarar una clase que implemente una interfaz es necesario utilizar la palabra reservada `implements` en la cabecera de declaración de la clase.

```
public class Clase implements IdentificadorInterfaz {
    // Cuerpo de la interfaz ...
}
```

Polimorfismo mediante interfaces

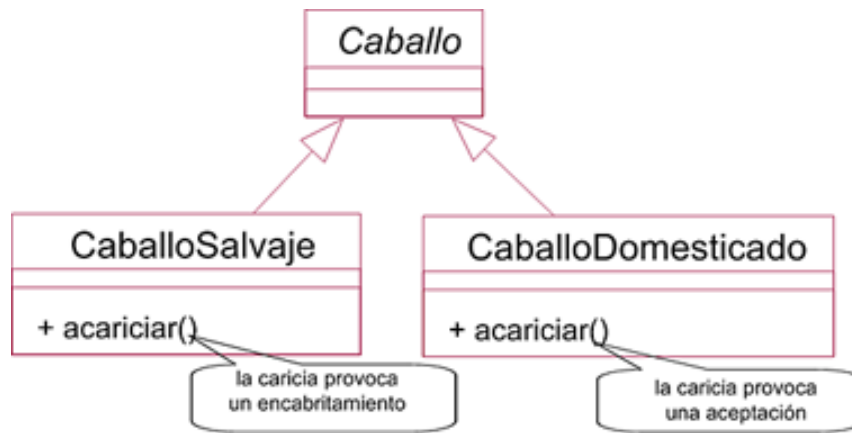
Igual que con las clases, se puede conseguir el polimorfismo con las interfaces porque **al implementar una interfaz podemos decir que el objeto de la clase es un objeto de la interfaz.**

El polimorfismo entra en juego cuando utilizamos una interfaz como tipo de referencia en lugar de una clase concreta. Esto nos permite escribir código genérico que puede trabajar con diferentes clases que implementan la misma interfaz.

Representando polimorfismo en un diagrama de clases

Tomemos la jerarquía de clases ilustrada en la siguiente figura. El método **acariciar** tiene un comportamiento diferente según si el caballo es una instancia de **CaballoSalvaje** o de **CaballoDomesticado**. En el primer caso, el comportamiento será un rechazo (que se traducirá en un encabritamiento) mientras que, en el segundo, el comportamiento será una aceptación.

Si consideramos la clase Caballo en su totalidad, tenemos un conjunto de caballos que **reaccionan de distinta manera al activarse el método acariciar.**



Al tratar los objetos como instancias de la interfaz, podemos invocar los métodos definidos en la interfaz sin conocer la implementación específica de la clase. Esto proporciona una gran flexibilidad, ya que podemos agregar nuevas clases que implementen la interfaz sin tener que modificar el código existente.

Principios de Diseño Orientado a Objetos

Un principio de diseño en Programación Orientada a Objetos (POO) es una regla o guía general que se utiliza para construir software de calidad y bien estructurado. Estos principios nos brindan directrices para organizar y diseñar nuestras clases y objetos de manera efectiva, promoviendo un código limpio, mantenible y fácil de entender.

Al seguir estos principios, podemos obtener múltiples beneficios en términos de organización del código, mantenibilidad, reutilización de código, flexibilidad y calidad del software en general.

Conceptos de mantenibilidad y reutilizabilidad

La mantenibilidad y la reutilizabilidad son dos conceptos fundamentales en el desarrollo de software que tienen un impacto significativo en la calidad y eficiencia de un sistema.

Mantenibilidad: se refiere a la **capacidad de un software para ser modificado, corregido, mejorado o adaptado de manera eficiente y efectiva** a lo largo del

tiempo. Un software fácilmente mantenible es aquel que permite realizar cambios sin introducir errores adicionales o afectar negativamente su funcionamiento.

La mantenibilidad es importante porque los sistemas de software evolucionan constantemente. Se requieren actualizaciones, correcciones de errores y mejoras en función de los cambios en los requisitos y las necesidades de los usuarios. Un software mantenible se caracteriza por un diseño claro, una estructura modular, un código limpio y una buena documentación. Esto facilita la comprensión del sistema, la identificación rápida de problemas y la implementación de cambios sin introducir nuevos errores.

Reutilizabilidad: se refiere a la capacidad de utilizar componentes, módulos o partes de software existentes en diferentes contextos o aplicaciones sin tener que reescribirlos desde cero. La reutilización de código es fundamental para evitar la duplicación de esfuerzos, ahorrar tiempo y recursos, y mejorar la eficiencia en el desarrollo de software.

La reutilizabilidad se logra mediante el diseño e implementación de componentes y bibliotecas que son independientes, modulares y fácilmente integrables en diferentes sistemas. Esto implica utilizar técnicas como la encapsulación, la abstracción y la creación de interfaces claras y flexibles.

Conceptos de cohesión y acoplamiento

La cohesión y el acoplamiento son dos conceptos fundamentales en el diseño de software orientado a objetos que se relacionan con la estructura interna de las clases y su interacción entre sí.

Cohesión: se refiere al **grado en que los elementos internos de una clase están relacionados** y trabajan juntos para lograr un objetivo común. Una clase con alta cohesión tiene métodos y atributos que están estrechamente relacionados y se centran en una sola responsabilidad o tarea.

Acoplamiento: mide el **grado en que una clase depende de otras clases**. Un acoplamiento bajo quiere decir que los cambios en una clase tienen un impacto mínimo en otras clases. Esto es deseable porque promueve la flexibilidad y la capacidad de cambio. Si las clases están débilmente acopladas, se pueden realizar modificaciones en una clase sin afectar a otras clases, lo que facilita la extensibilidad y la adaptación del sistema a nuevas necesidades.

Principios SOLID

Los principios SOLID son un conjunto de principios de diseño de software que promueven la modularidad, la flexibilidad y la facilidad de mantenimiento.

- **Principio de Responsabilidad Única (SRP):** Cada clase debe tener una única responsabilidad y motivo para cambiar. Este principio es importante porque una clase con una responsabilidad única es más fácil de entender, mantener y probar. Al tener una única responsabilidad, los cambios en los requisitos o funcionalidades relacionados con esa responsabilidad específica afectarán únicamente a esa clase, lo que reduce el riesgo de errores y evita efectos secundarios no deseados en otras partes del sistema.
- **Principio de Abierto/Cerrado (OCP):** Las clases deben estar abiertas para la extensión pero cerradas para la modificación.
- **Principio de Sustitución de Liskov (LSP):** Las instancias de una clase derivada deben poder ser utilizadas en lugar de instancias de la clase base sin alterar la corrección del programa.
- **Principio de Segregación de Interfaces (ISP):** Los clientes no deben depender de interfaces que no utilicen. Las interfaces deben ser específicas y no deben tener métodos innecesarios.
- **Principio de Inversión de Dependencia (DIP):** Los módulos de alto nivel no deben depender de módulos de bajo nivel, ambos deben depender de abstracciones. Además, los detalles deben depender de abstracciones y no al revés.

Cierre

En esta lección hemos aprendido sobre el polimorfismo y los principios básicos de diseño en Java. El polimorfismo nos permite tratar objetos de diferentes tipos de manera uniforme, lo cual brinda flexibilidad y extensibilidad a nuestro código. Aprendimos a implementar interfaces y a utilizar la herencia como forma de reutilización de código.

Los principios de diseño nos ofrecen pautas para crear software modular y mantenible. Estos principios incluyen la responsabilidad única, la extensibilidad sin modificar el código existente, la sustitución segura de objetos, la segregación de interfaces y la inversión de dependencias.

Aplicar el polimorfismo y los principios de diseño nos ayuda a construir sistemas de software más robustos y fáciles de mantener. Estos conceptos son fundamentales para desarrollar código eficiente y de alta calidad en Java.

¡Nos vemos de nuevo, más adelante! 

Referencias

- López Quesada, J. A. (s. f.). Herencia [Actividad de página 9]. En Curso de Programación en Java (IAW): Actividades Alumnos. Universidad de Murcia. Recuperado el 25 de julio de 2025 de https://www.um.es/web/dis~lopezquesada/documentos/IES_1415/IAW/cursos/UT3/ActividadesAlumnos/java7/paginas/pag9.html
- López Quesada, J. A. (s. f.). Arquitectura Java: Tipos de herencia en Java [Recurso educativo]. Universidad de Murcia. Recuperado el 25 de julio de 2025, de https://www.um.es/web/dis~lopezquesada/documentos/IES_1415/IAW/cursos/UT3/ActividadesAlumnos/java7/paginas/pag9.html
- Garro, A. (s. f.). Capítulo 18: Interfaces [Recurso web]. En Java por Arkaitz Garro. Recuperado el 25 de julio de 2025, de <https://www.arkaitzgarro.com/java/capitulo-18.html>
- Dar10-comyr. (2014, 4 de octubre). Polimorfismo e Interface [Entrada de blog]. Una Taza de Java. Recuperado el 25 de julio de 2025, de <https://dar10comyr.blogspot.com/2014/10/polimorfismo-e-interface.html>
- If Geek, Then. (s. f.). Polimorfismo en Java [Entrada web]. NTT Data. Recuperado el 25 de julio de 2025, de <https://ifgeekthen.nttdata.com/s/post/polimorfismo-en-java-programacion-orientada-objetos-MCIU2TZFKR6FFIJMDQQASC7CU75I?language=es>
- If Geek, Then. (s. f.). Clases abstractas e Interfaces [Entrada web]. NTT Data. Recuperado el 25 de julio de 2025, de <https://ifgeekthen.nttdata.com/s/post/clases-abstractas-e-interfaces-MCRLZG2CKOIFF2JKPMO7XLKRWJAI?language=es>
- freeCodeCamp Español. (s. f.). Los principios SOLID de programación orientada a objetos explicados en Español sencillo [Artículo web]. Recuperado el 25 de julio de 2025, de <https://www.freecodecamp.org/espanol/news/los-principios-solid-explicados-en-espanol/>

¡Muchas gracias!

Nos vemos en la próxima lección

